

The topics discussed in this chapter were either too large or far too removed from the mainstream C programming for inclusion in the earlier chapters. These topics provide certain useful programming features, and could prove to be of immense help in certain programming strategies. In this chapter, we would examine enumerated data types, the typedef keyword, typecasting, bit fields, function pointers, functions with variable number of arguments and unions.

Enumerated Data Type

The enumerated data type gives you an opportunity to invent your own data type and define what values the variable of this data type can take. This can help in making the program listings more readable, which can be an advantage when a program gets complicated or when more than one programmer would be working on it. Using enumerated data type can also help you reduce programming errors.

As an example, one could invent a data type called `mar_status` which can have four possible values—single, married, divorced or widowed. Don't confuse these values with variable names; married, for instance, has the same relationship to the variable `mar_status` as the number 15 has with a variable of type `int`.

The format of the `enum` definition is similar to that of a structure. Here's how the example stated above can be implemented:

```
enum mar_status
{
    single, married, divorced, widowed
};
enum mar_status person1, person2;
```

Like structures, this declaration has two parts:

- The first part declares the data type and specifies its possible values. These values are called 'enumerators'.
- The second part declares variables of this data type.

Now we can give values to these variables:

```
person1 = married ;
person2 = divorced ;
```

Remember, we can't use values that aren't in the original declaration.

Thus, the following expression would cause an error:

```
person1 = unknown ;
```

Internally, the compiler treats the enumerators as integers. Each value on the list of permissible values corresponds to an integer, starting with 0. Thus, in our example, single is stored as 0, married is stored as 1, divorced as 2 and widowed as 3.

This way of assigning numbers can be overridden by the programmer by initializing the enumerators to different integer values as shown below.

```
enum mar_status
{
    single = 100, married = 200, divorced = 300, widowed = 400
};
enum mar_status person1, person2 ;
```

Uses of Enumerated Data Type

Enumerated variables are usually used to clarify the operation of a program. For example, if we need to use employee departments in a payroll program, it makes the listing easier to read if we use